

MoQ Optimization for Point-cloud Volumetric Video Streaming

Utilizing the Forward parameter & Subscriber Priority

2026/10/02 Kota Yatagai @ MoQ Meetup Japan

Agenda

- Introduction of point-cloud volumetric video
- FoV-adaptive streaming
- Building FoV-adaptive streaming with MoQ
- Optimizing MoQ for better experience
 - Utilizing the Forward parameter to reduce tail-frame drops
 - Mapping segment position and angle to the Subscriber Priority

Speaker: Kota Yatagai



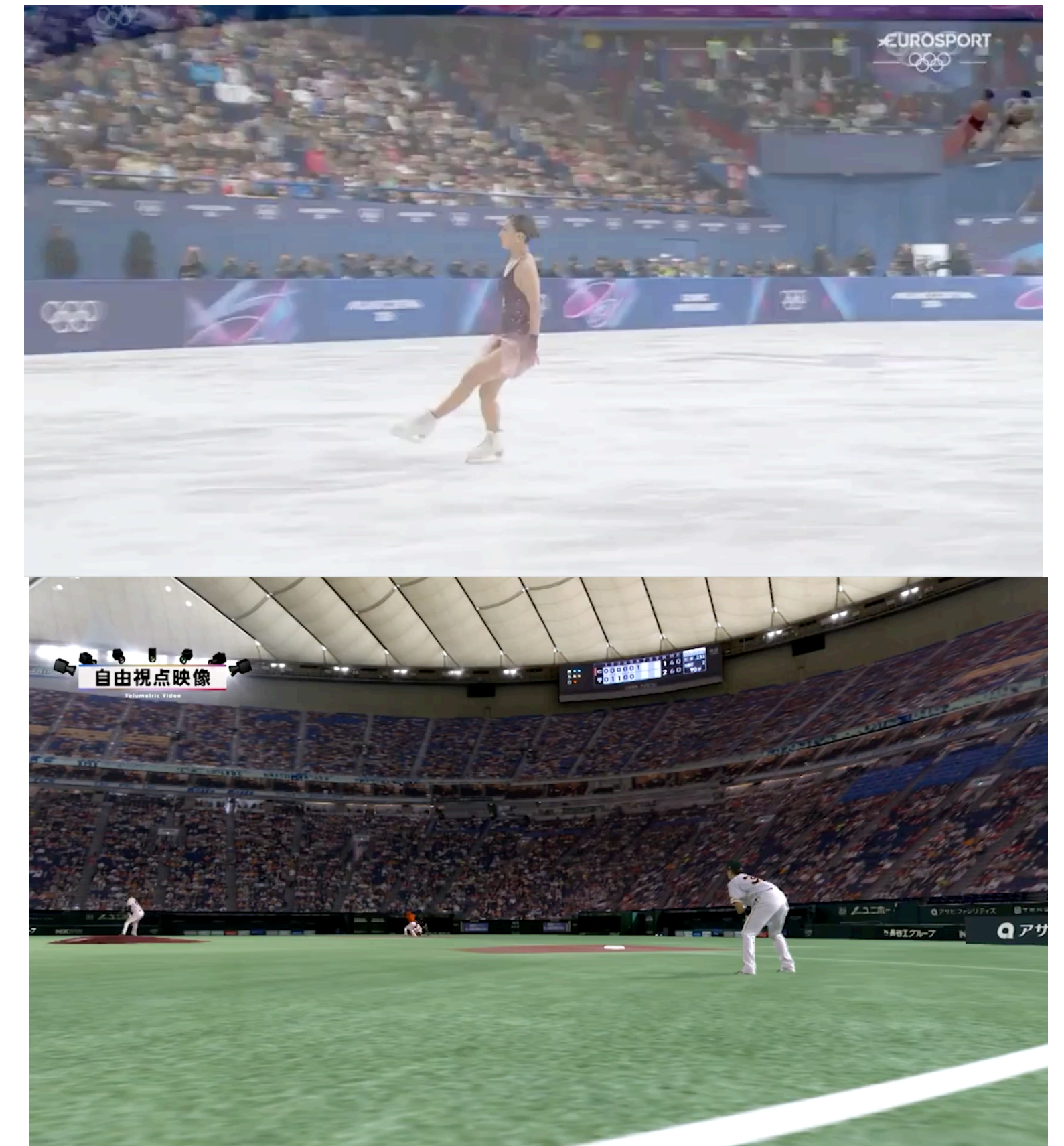
X: @kota_yata
LinkedIn: /in/kota-yata

- ▶ Last-year undergraduate at Keio University SFC
- ▶ Research theme: Volumetric video streaming
- ▶ OSS contributions:
 - ▶ jitsi/jitsi-videobridge (funded by Google Summer of Code)
 - ▶ wireshark/wireshark
 - ▶ MoQ things like imquic, moq-wasm, MOQtail etc.
- ▶ Developing Moqtopus ([yatagai-mm/moqtopus](https://github.com/yatagai-mm/moqtopus))
 - ▶ MoQ client library in C++
 - ▶ Lightweight, kinda fast



Point-cloud volumetric video, and sports!

- ▶ 3D video made of a bunch of points
 - ▶ Each point has coordinates and color
- ▶ It could be transformed into mesh, 3DGS etc.
- ▶ Bare point clouds are used for real-time rendering/presentations for latency reasons



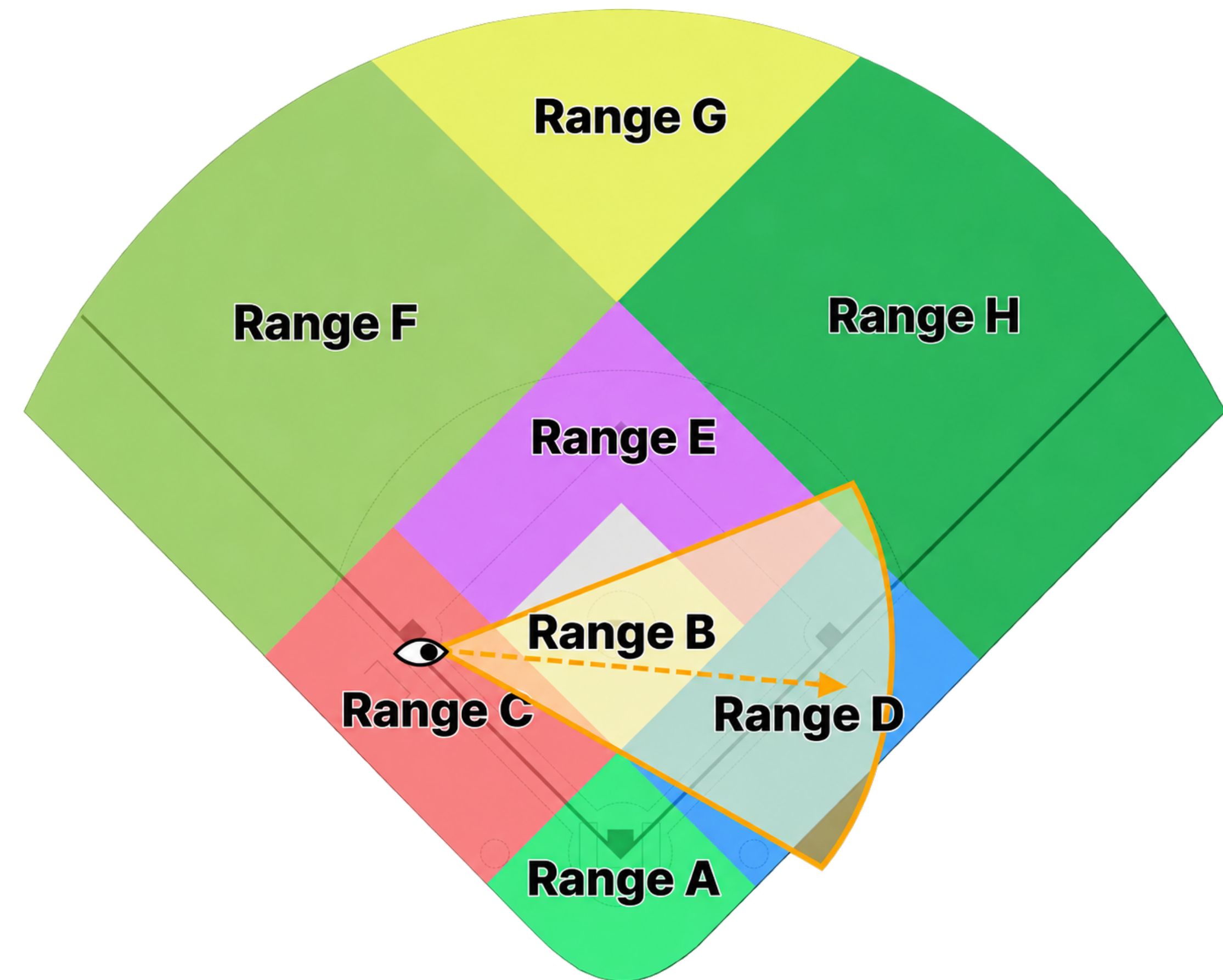
- ▶ 150 camera around the field at Tokyo Dome
- ▶ Generates VV replay within 3 seconds

Point-cloud VV is heavy

- Currently deployed services do not live-stream the VV
 - They just capture/render VV in place and generate 2D replay video
- A high-quality frame of the whole scene: >30MB
 - 14.4Gbps@60fps
- A mid-quality frame of the whole scene: ~5MB
 - 2.4Gbps@60fps
- These won't go through access networks (WiFi or 5G)
- A lot of research going on to make volumetric video streaming possible.
 - (And I want free-viewpoint baseball game at home!!)

FoV-adaptive Streaming

- ▶ The sender separates a scene into multiple segments
- ▶ On the viewer side, predict its field-of-view (FoV) a few frames ahead
- ▶ Request the segments the predicted FoV intersects with
- ▶ Pretty much all the state-of-the-art volumetric video research hires this approach
 - ▶ Originally used in 360° video streaming
- ▶ **Requires real-time FoV prediction and low-latency transport**



Viewer will be looking at the first base from third base at the next n frame. The predicted FoV intersects with C, B, D, E and H, so request them

Is MoQ Suitable for FoV-adaptive Streaming?

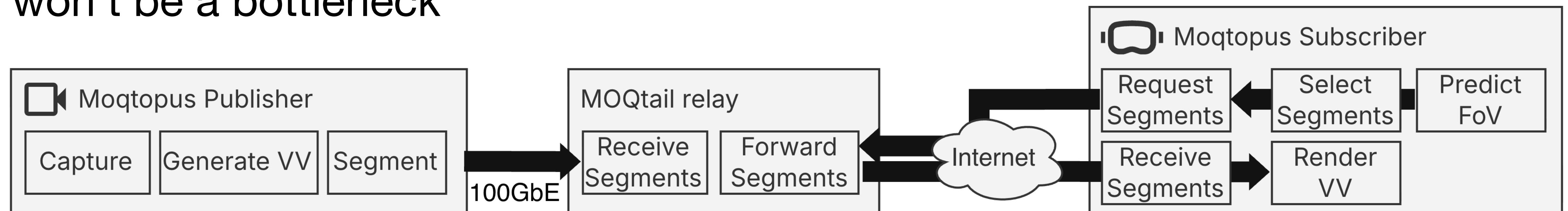
Is MoQ suitable for FoV-adaptive streaming?

- ▶ **YES**

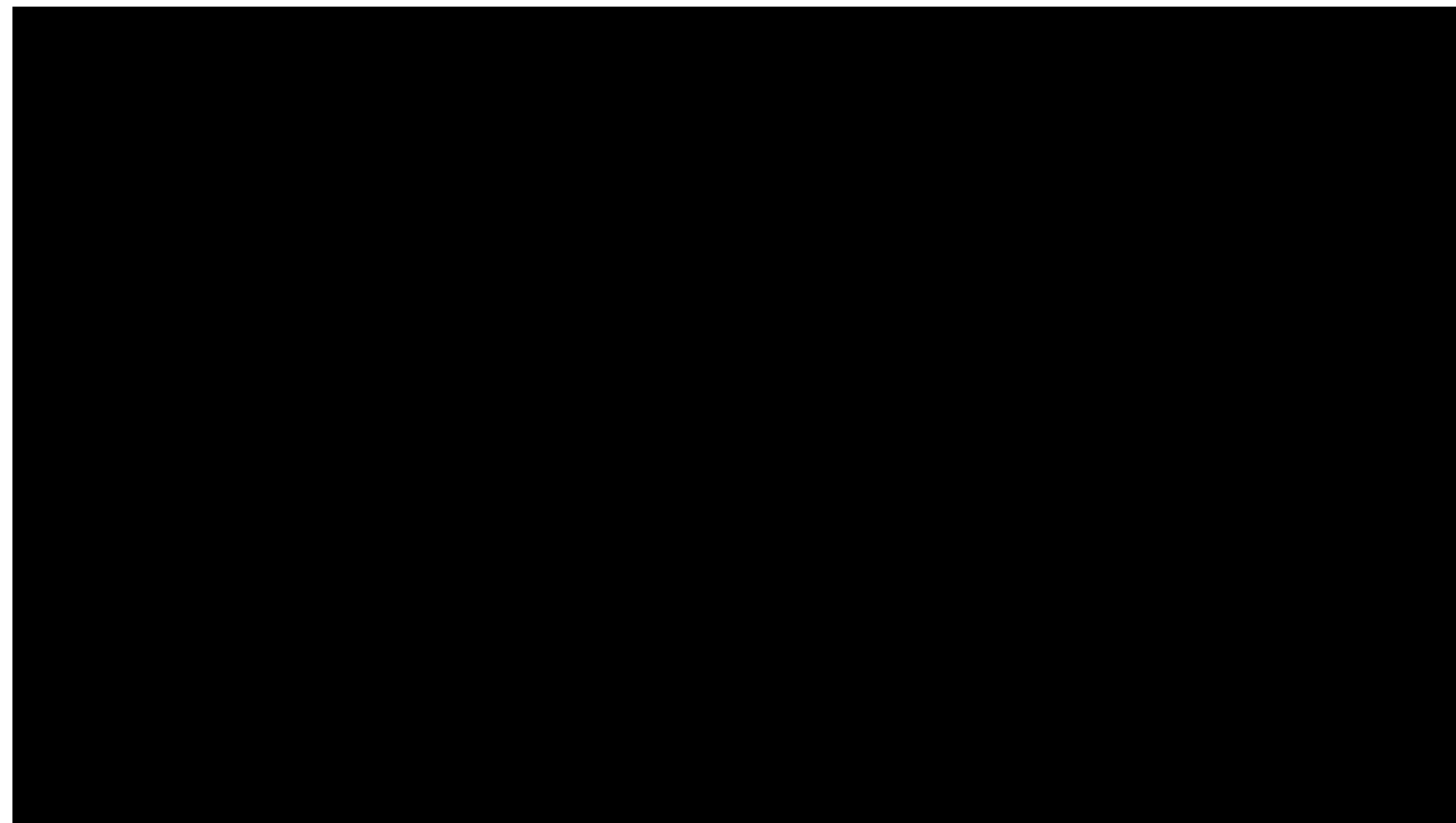
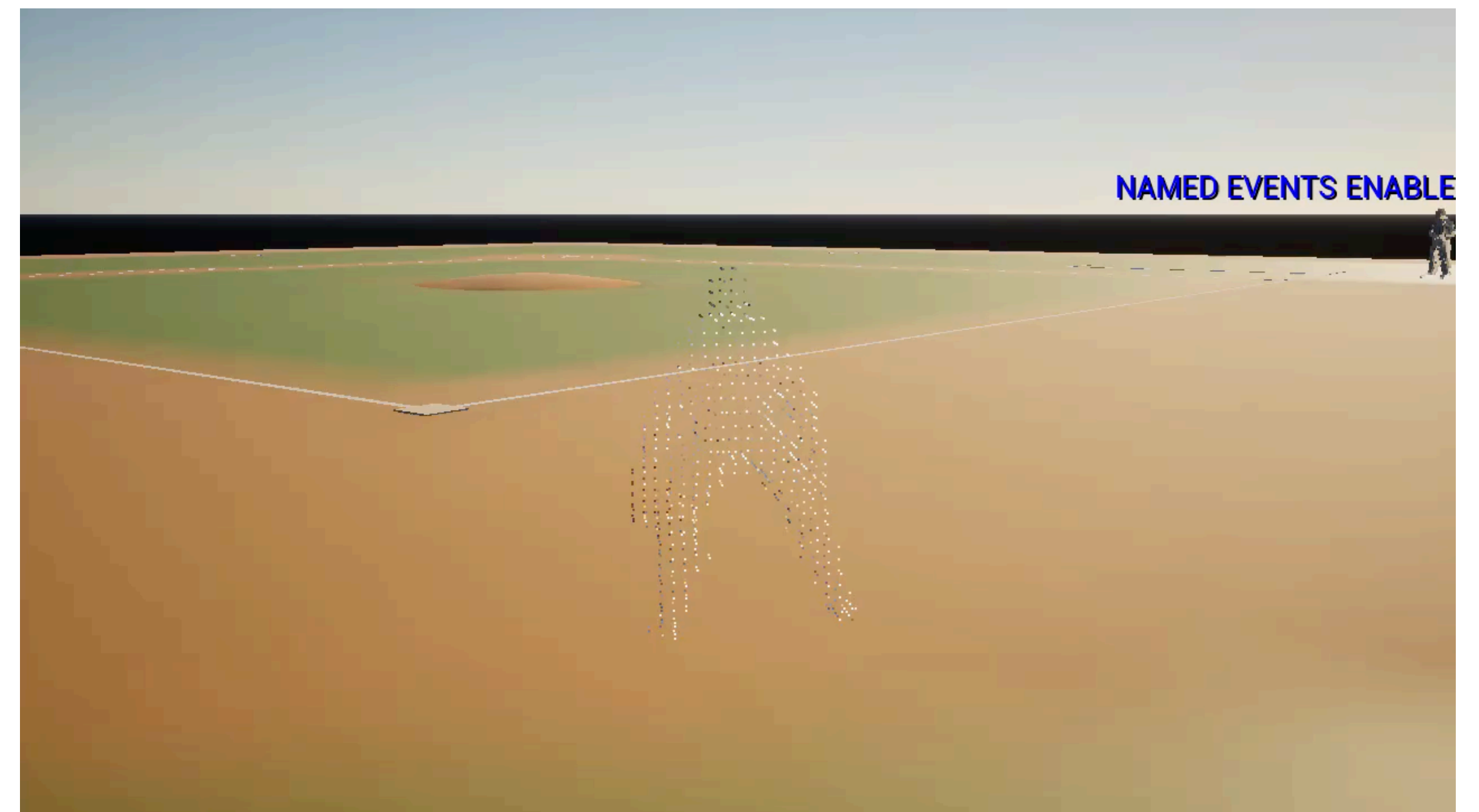
- ▶ HLS/DASH would require a total redesign to enable high-frequency prediction
- ▶ WebRTC is nice if you are sending video or audio
 - ▶ You have to use DataChannel for point clouds, and then you need a custom media protocol on top of it
 - ▶ Request/reply mechanism, control plane, prioritization...
- ▶ MoQ is a media-agnostic pub/sub protocol
 - ▶ low-latency, scalable media transmission
 - ▶ It will be the closest path to build VV streaming pipeline

Build with MoQ

- ▶ A MoQ track corresponds to a segment
- ▶ **Moqtopus** for the publisher/subscriber, **MOQtail** for the relay
- ▶ Publisher repeatedly sends 7s of baseball point-cloud VV
- ▶ Subscriber SUBSCRIBES the track corresponds to the segment within a predicted FoV (5 frames ahead)
 - ▶ When it's out of the FoV, unsubscribe it (STOP_SENDING)
- ▶ The publisher and relay are connected with 100GbE so upstream bandwidth won't be a bottleneck

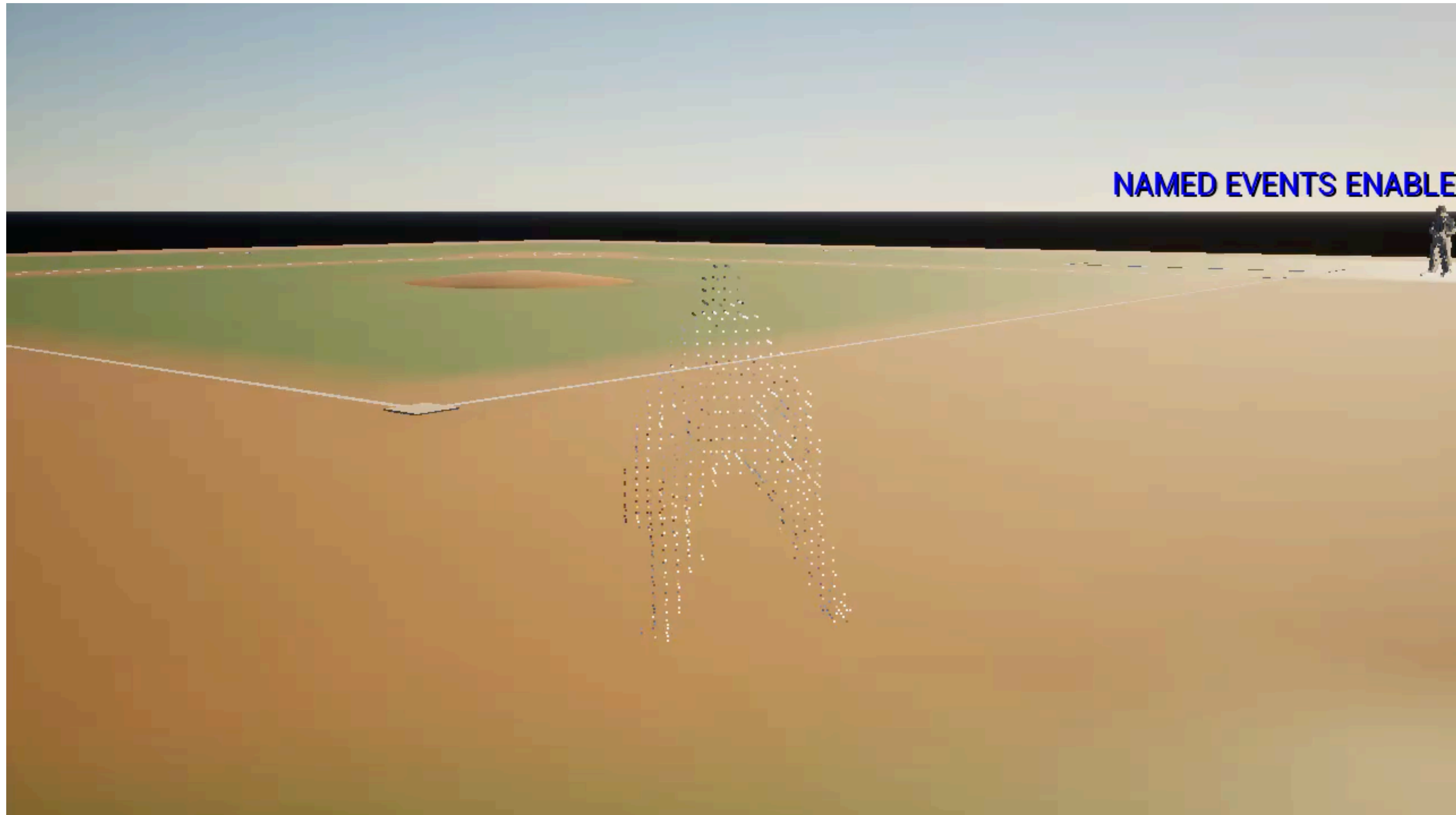


It worked!



Time to optimize

Frame drops observed

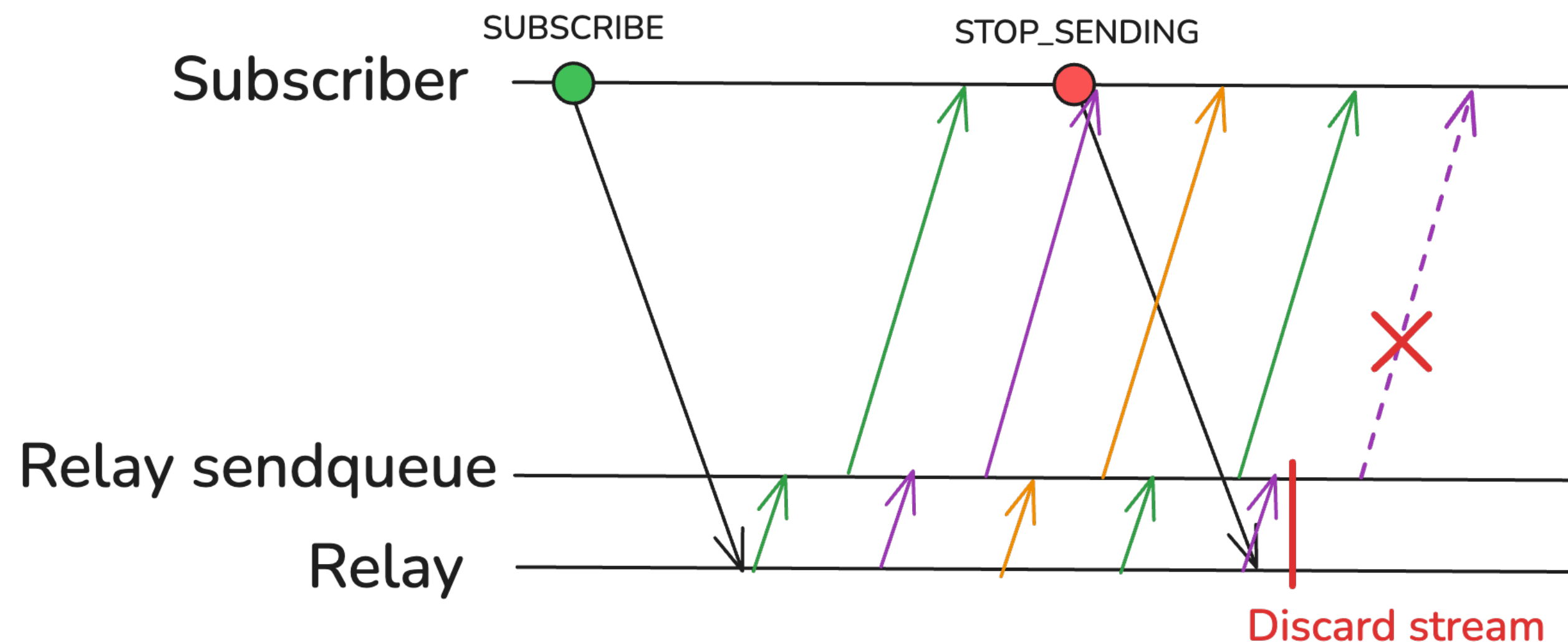


Relay CPU wasn't saturated

- ▶ My hypothesis was the relay CPU was saturated because of frequent subscription churn and its state management
- ▶ MOQtail handled subscription churn perfectly
 - ▶ Subscription/unsubscription could happen at every frame in the worst case
- ▶ I gave MOQtail (or tokio) 10 cores, but it only used 3 cores at maximum
 - ▶ Even when scaling the number of subscribers to 50
- ▶ The time from SUBSCRIBE reception to the next write_all to the corresponding stream was only few microseconds
- ▶ It was not the relay causing the frame drop

RESET_STREAM resets the stream!

- ▶ In recent version of MoQ, Subscriber UNSUBSCRIBES by sending STOP_SENDING on the bidirectional request stream.
 - ▶ (We don't UNSUBSCRIBE anymore btw)
- ▶ Then the relay sends RESET_STREAM to all streams in the subscription, cancelling the stream right away
- ▶ Objects still in the send queue could be discarded

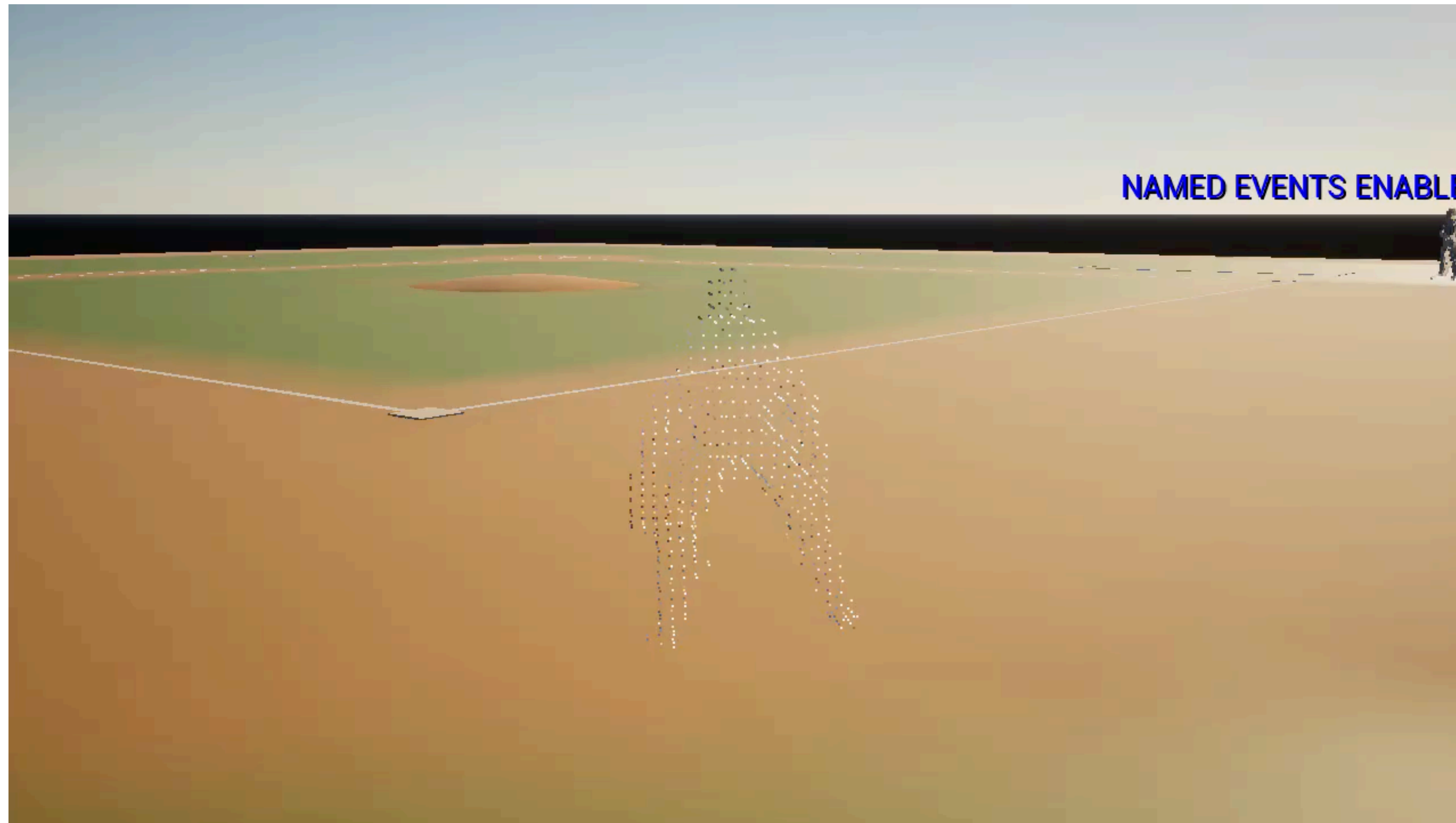


# of lost frames during 20s run	40
# of lost frames that are the last two in a stream during the same run	36

Using the **Forward** parameter instead

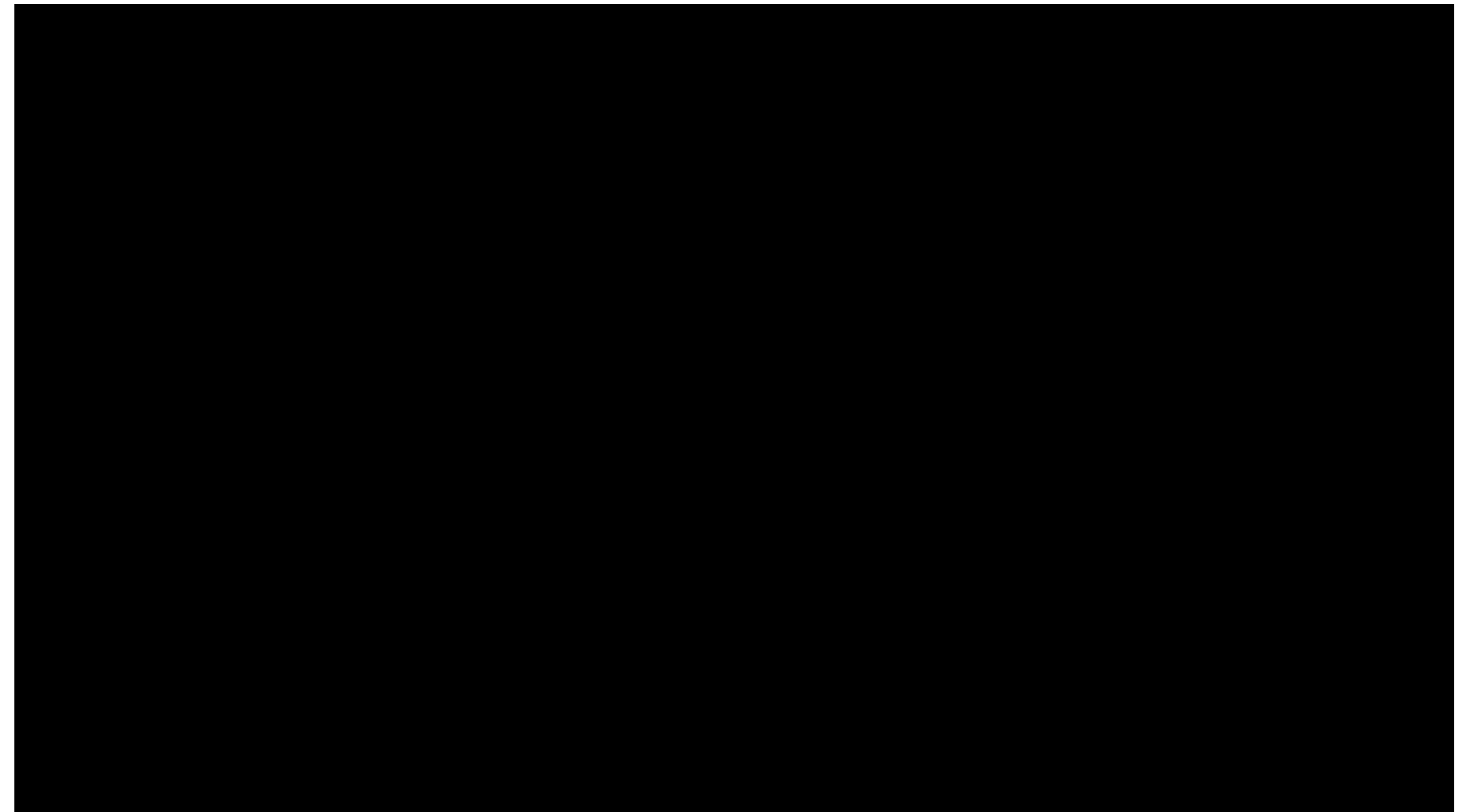
- ▶ **Forward** is a binary parameter in a subscription
- ▶ `REQUEST_UPDATE(forward=0)` tells the relay to stop forwarding objects in the subscription
- ▶ The relay retains the subscription so there's no stream reset happening
- ▶ It was originally defined to pre-warm the subscription
- ▶ Also useful for frequent track switching and prediction like this workload.

No dropping of tail frames with Forward



SUBSCRIBE

Reception rate: 94.40%



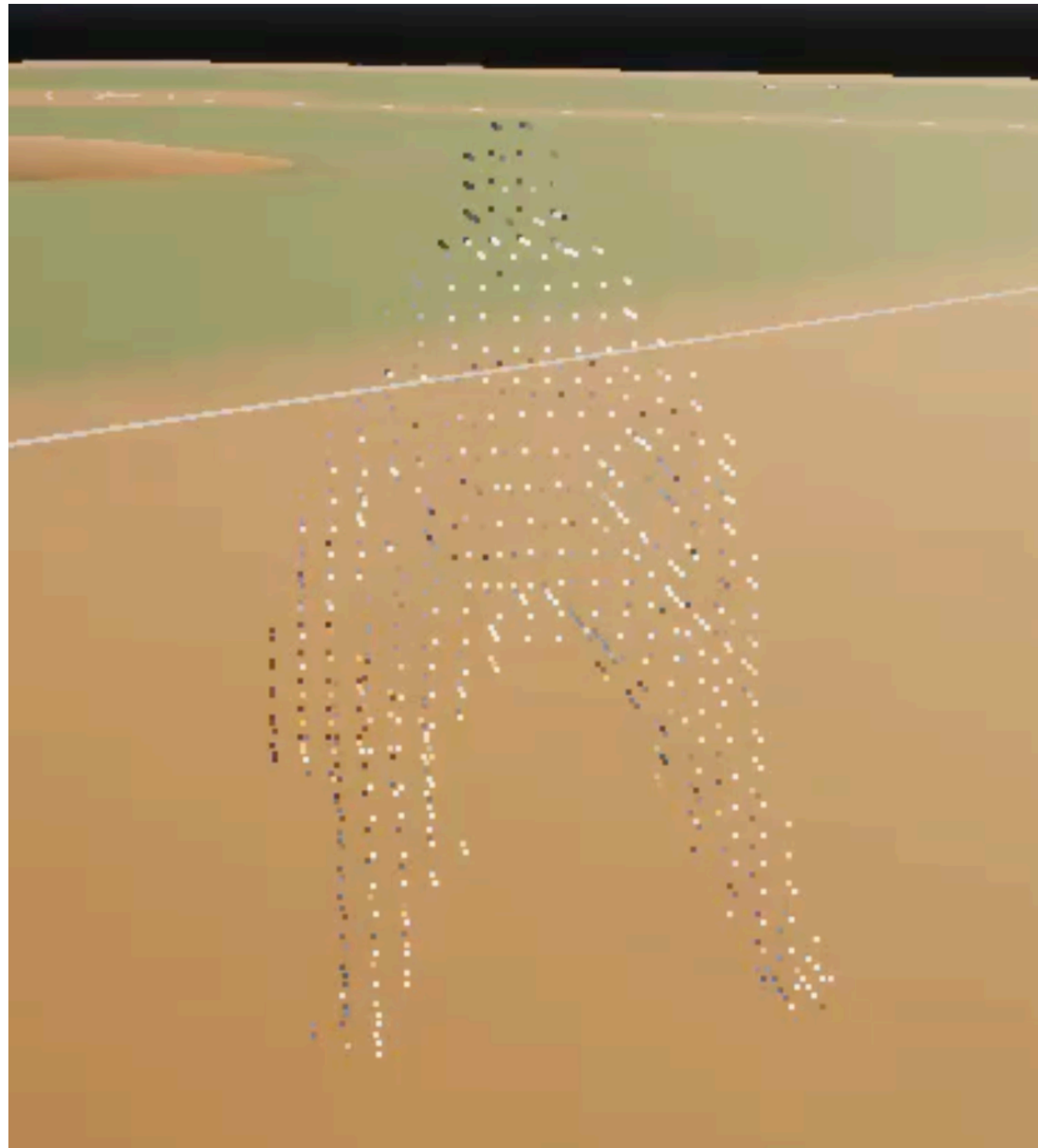
Forward

Reception rate: 99.98%

Everything's good now?..

Next Problem: viewing experience

- ▶ You only get “objects” shaping like a human if you have poor WiFi



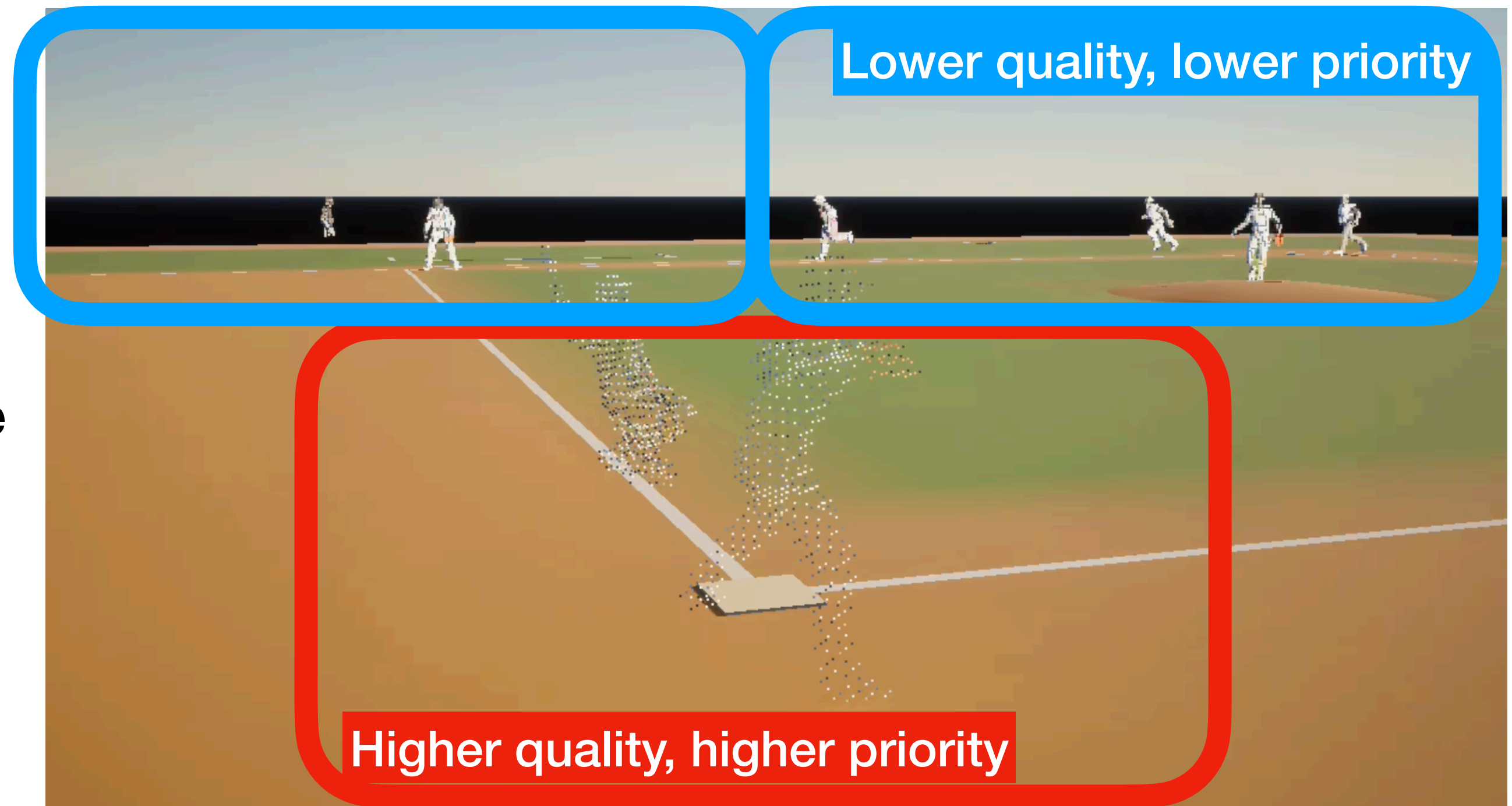
- ▶ A high-quality frame of the whole scene: >30MB
 - 14.4Gbps@60fps
 - ~1.4Gbps per segment
- ▶ A mid-quality frame of the whole scene: ~5MB
 - 2.4Gbps@60fps
 - ~240Mbps per segment
- ▶ If you are watching baseball, you are likely to watch the whole infield most of the time
 - Reduction in bandwidth consumption is limited

← This is what I got with 100Mbps

Request higher quality for closer segments

- For the segment the predicted position is in, request mid-quality track
- For other frames, request low-quality tracks
- The closer the segment is to the predicted position, the higher its quality

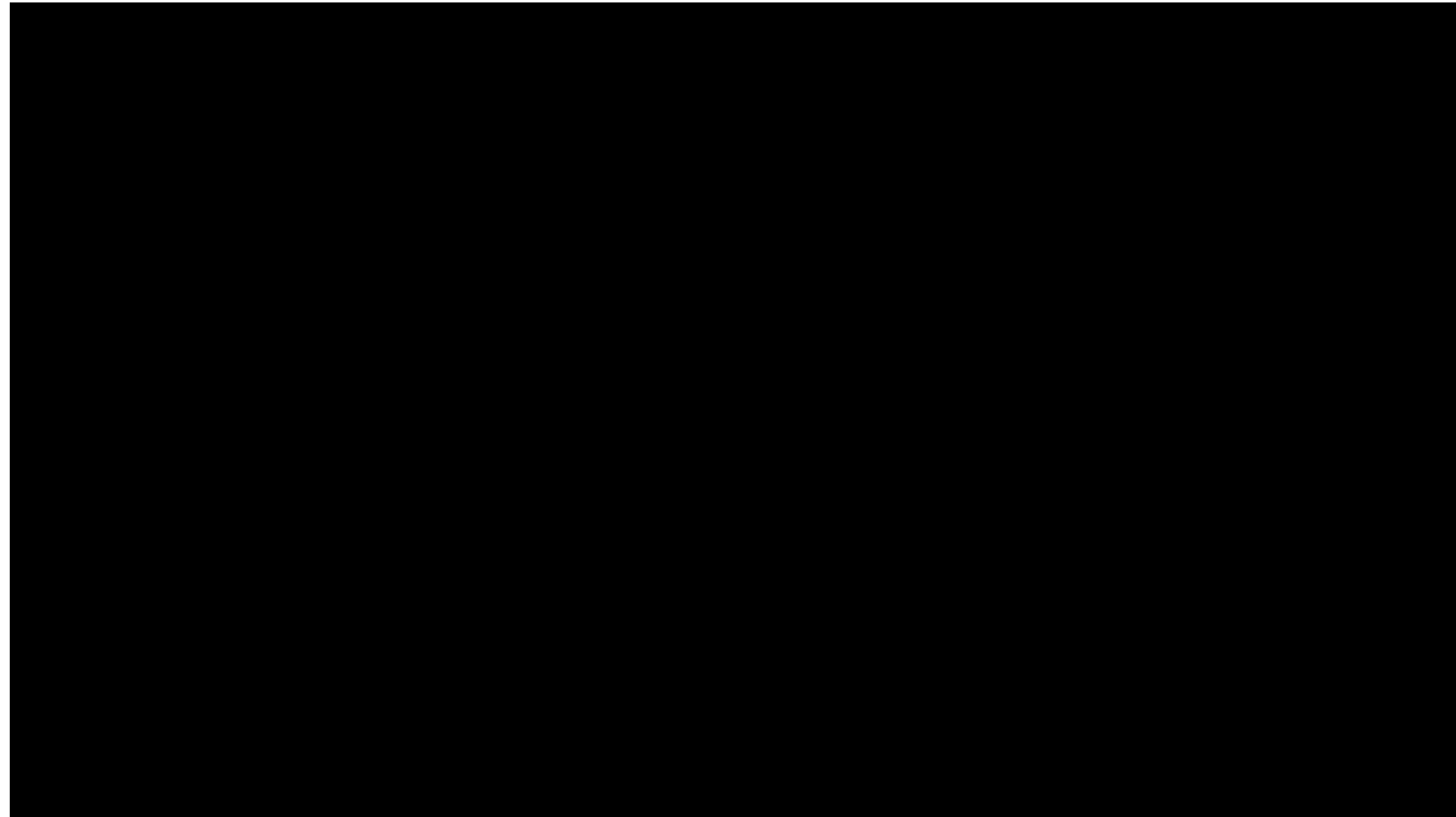
Improve closer segments because far segments already look fine



Mapping distance/angle to Subscriber Priority

- ▶ **Subscriber Priority** is per-subscription priority set by a subscriber
- ▶ Relay decides which object to send next based on the priority
 - ▶ Subscriber Priority takes precedence over Publisher Priority and Group Order
- ▶ I scored segments in a predicted FoV based on the distance/angle from the predicted position
 - ▶ $\text{score} = \text{round}(7/2(\text{normalized_distance} + \text{normalized_angle})) * 32$
 - ▶ ↑ Basically separates segments into 8 priorities
- ▶ Closer the higher priority, the closer to the center the higher priority

Closer segments looks clearer, with the same bandwidth



No Priority, low quality for every segments



Priority, mid quality for the closest segment

- Send buffer, recv window and maximum number of streams are radically extended
- Renderer's queue is also optimized to wait longer for higher priority objects
- Scoring is a design choice (some might want to see everything with lower quality)

Conclusion

- ▶ Forward was useful for pre-loading predicted frames
- ▶ Subscriber Priority was useful for optimizing the closer segments
- ▶ Overall, FoV-adaptive streaming of volumetric video seems to work over MoQ

